

# Li Chao Tree

## 李超线段树

### 引入

▼ [洛谷 4097 \[HEOI2013\]Segment](#)

要求在平面直角坐标系下维护两个操作（强制在线）：

1. 在平面上加入一条线段。记第  $i$  条被插入的线段的标号为  $i$ ，该线段的两个端点分别为  $(x_i, y_i), (x_i', y_i')$ 。
2. 给定一个数  $x$ ，询问与直线  $x = x$  相交的线段中，交点纵坐标最大的线段的编号（若有多条线段与查询直线的交点纵坐标都是最大的，则输出编号最小的线段）。特别地，若不存在线段与给定直线相交，输出  $-1$ 。

数据满足：操作总数  $n \leq 10^5$ ， $x_i, x_i' \in [-10^9, 10^9]$ ， $y_i, y_i' \in [-10^9, 10^9]$ 。

我们发现，传统的线段树无法很好地维护这样的信息。这种情况下，**李超线段树** 便应运而生。

### 过程

我们可以把任务转化为维护如下操作：

- 加入一个一次函数，定义域为  $[x_l, x_r]$ ；
- 给定  $x$ ，求定义域包含  $x$  的所有一次函数中，在  $x$  处取值最大的那个，如果有多个函数取值相同，选编号最小的。

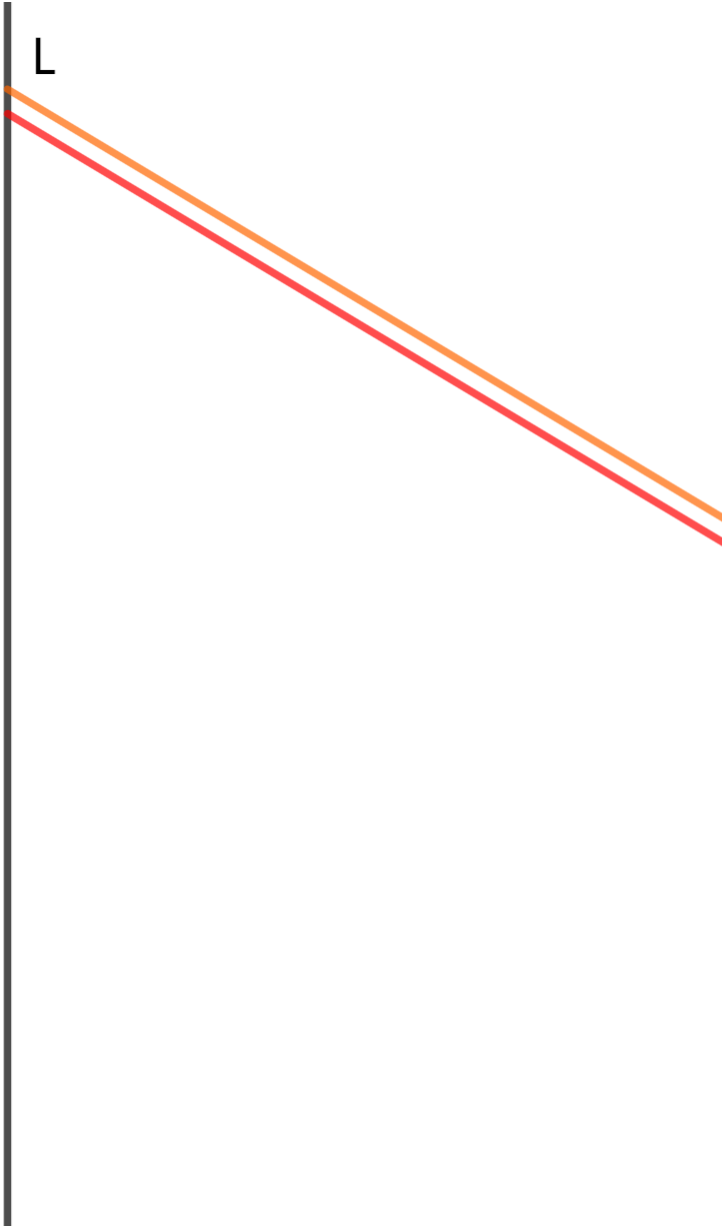
▼ 注意

当线段垂直于  $y$  轴时，会出现除以零的情况。假设线段两端点分别为  $(x_l, y_l)$  和  $(x_r, y_r)$ ，则插入定义域为  $[x_l, x_r]$  的一次函数  $y = \frac{y_r - y_l}{x_r - x_l}(x - x_l) + y_l$ 。

看到区间修改，我们按照线段树解决区间问题的常见方法，给每个节点一个懒标记。每个节点的懒标记都是一条线段，记为  $l$ ，表示要用  $l$  更新该节点所表示的整个区间。

现在我们需要插入一条线段  $s$ ，考虑某个被新线段  $s$  完整覆盖的线段树区间。若该区间无标记，直接打上用该线段更新的标记。

如果该区间已经有标记了，由于标记难以合并，只能把标记下传。但是子节点也有自己的标记，也可能产生冲突，所以我们要递归下传标记。





如图，按新线段 取值是否大于原标记，我们可以把当前区间分为两个子区间。其中 肯定有一个子区间被左区间或右区间完全包含，也就是说，在两条线段中，肯定有一条线段，只可能成为左区间的答案，或者只可能成为右区间的答案。我们用这条线段递归更新对应子树，用另一条线段作为懒标记更新整个区间，这就保证了递归下传的复杂度。当一条线段只可能成为左或右区间的答案时，才会被下传，所以不用担心漏掉某些线段。

具体来说，设当前区间的中点为  $mid$ ，我们拿新线段 在中点处的值与原最优线段 在中点处的值作比较。

如果新线段 更优，则将 和 交换。那么现在考虑在中点处 不如 优的情况：

- 1. 若在左端点处 更优，那么 和 必然在左半区间中产生了交点，只有在左区间才可能优于，递归到左儿子中进行下传；
- 2. 若在右端点处 更优，那么 和 必然在右半区间中产生了交点，只有在右区间才可能优于，递归到右儿子中进行下传；
- 3. 若在左右端点处 都更优，那么 不可能成为答案，不需要继续下传。

除了这两种情况之外，还有一种情况是 和 刚好交于中点，在程序实现时可以归入中点处 不如 优的情况，结果会往 更优的一个端点进行递归下传。

最后将 作为当前区间的懒标记。

下传标记：

▼ 实现

```

1 constexpr double eps = 1e-9;
2
3 int cmp(double x, double y) { // 因为用到了浮点数, 所以会有精度误差
4     if (x - y > eps) return 1;
5     if (y - x > eps) return -1;
6     return 0;
7 }
8
9 //...
10
11 void upd(int root, int cl, int cr, int u) { // 对线段完全覆盖到的区间进行修改
12     int &v = s[root], mid = (cl + cr) >> 1;
13     int bmid = cmp(calc(u, mid), calc(v, mid));
14     if (bmid == 1 || (!bmid && u < v)) // 在此题中记得判线段编号
15         swap(u, v);
16     int bl = cmp(calc(u, cl), calc(v, cl)), br = cmp(calc(u, cr), calc(v, cr));
17     if (bl == 1 || (!bl && u < v)) upd(root << 1, cl, mid, u);
18     if (br == 1 || (!br && u < v)) upd(root << 1 | 1, mid + 1, cr, u);
19     // 上面两个 if 的条件最多只有一个成立, 这保证了李超树的时间复杂度
20 }

```

拆分线段：

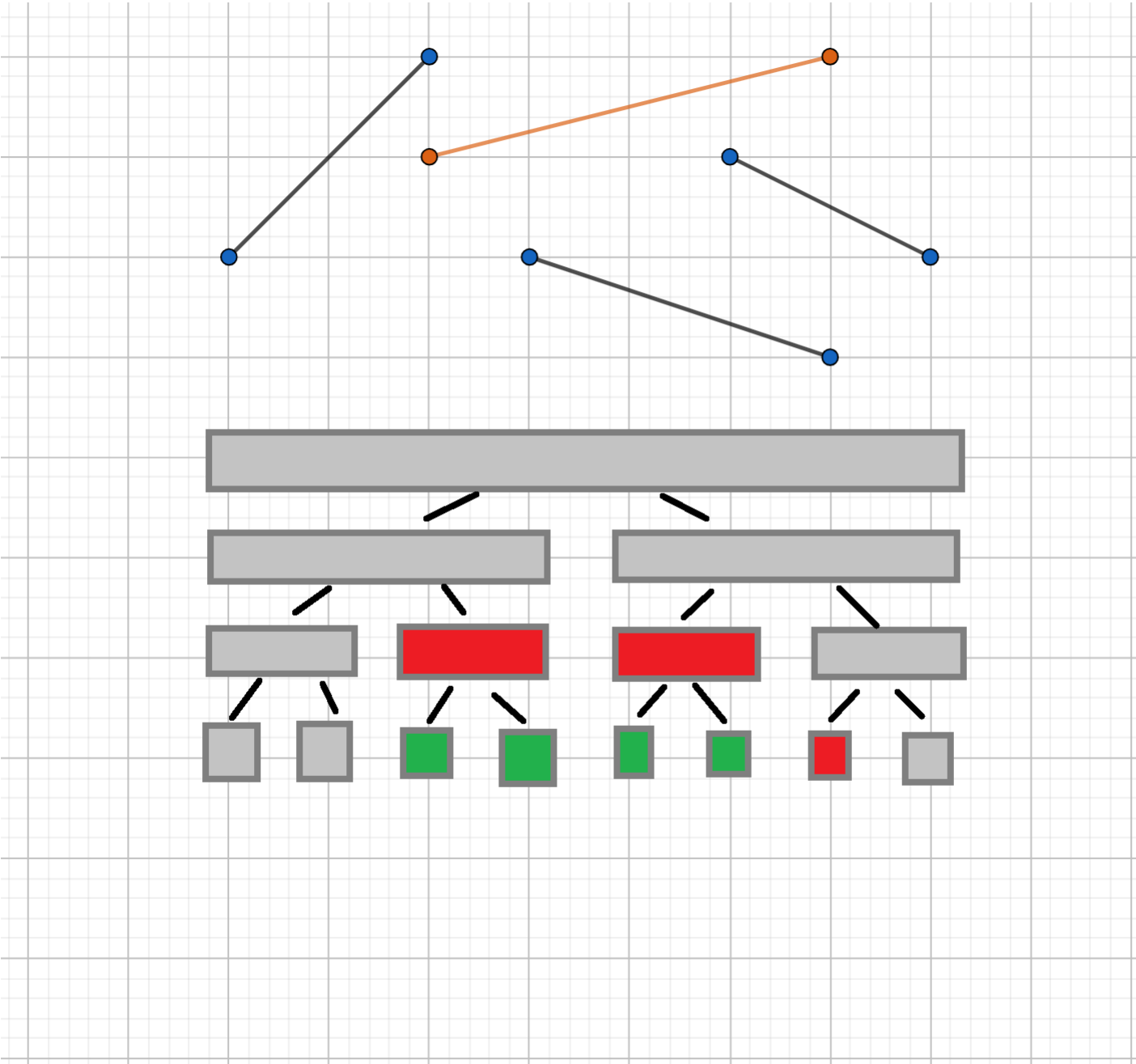
#### ▼ 实现

```

1 void update(int root, int cl, int cr, int l, int r,
2             int u) { // 定位插入线段完全覆盖到的区间
3     if (l <= cl && cr <= r) {
4         upd(root, cl, cr, u); // 完全覆盖当前区间, 更新当前区间的标记
5         return;
6     }
7     int mid = (cl + cr) >> 1;
8     if (l <= mid) update(root << 1, cl, mid, l, r, u); // 递归拆分区间
9     if (mid < r) update(root << 1 | 1, mid + 1, cr, l, r, u);
10 }

```

注意懒标记并不等价于在区间中点处取值最大的线段。



如图，加入黄色线段后，只有红色节点的标记被更新，而绿色节点的标记还未被改变。但在第二、三、四个绿色区间的中点处显然黄色线段取值最大。

查询时，我们可以利用标记永久化思想，在包含 的所有线段树区间（不超过 个）的标记线段中，比较得出最终答案。

查询：

▼ 实现

```
1 pdi query(int root, int l, int r, int d) { // 查询
2   if (r < d || d < l) return {0, 0};
3   int mid = (l + r) >> 1;
4   double res = calc(s[root], d);
5   if (l == r) return {res, s[root]};
6   return pmax({res, s[root]}, pmax(query(root << 1, l, mid, d),
7                                     query(root << 1 | 1, mid + 1, r, d)));
8 }
```

根据上面的描述，查询过程的时间复杂度显然为 ，而插入过程中，我们需要将原线段拆分到 个区间中，对于每个区间，我们又需要花费 的时间递归下传，从而插入过程的时间复杂度为 。

► [\[HEOI2013\]Segment](#) 参考代码

## 合并

类似于普通线段树的合并，我们定义以下过程来将两个李超线段树节点 合并，并以 作为新的根。

1. 如果 为空，结束过程。
2. 如果 为空，将 复制给 。
3. 将 对应线段插入到 为根的子树。
4. 递归将 的左右子树对应合并。

若合并若干李超线段树涉及的总点数为 ，则该过程的复杂度为 ：对于任意线段在树上对应的节点，我们要么使其深度，要么直接从树上删除，这两个操作的代价都是 的，而

每个点深度至多为  $\log n$ ，于是复杂度如上。

▼ 实现

```
1 void upd(int &root, int cl, int cr,
2         int u) { // 涉及多棵李超线段树合并，使用动态开点。
3     static int idx = 0;
4     if (!root) {
5         s[root = ++idx] = u;
6         return;
7     }
8     int &v = s[root], mid = (cl + cr) >> 1;
9     int bmid = cmp(calc(u, mid), calc(v, mid));
10    if (bmid == 1 || (!bmid && u < v)) swap(u, v);
11    int bl = cmp(calc(u, cl), calc(v, cl)), br = cmp(calc(u, cr), calc(v, cr));
12    if (bl == 1 || (!bl && u < v)) upd(ls[root], cl, mid, u);
13    if (br == 1 || (!br && u < v)) upd(rs[root], mid + 1, cr, u);
14 }
15
16 int merge(int &u, int &v, int l, int r) {
17     if (!u || !v) {
18         return u + v;
19     }
20     if (l == r) {
21         int b = cmp(calc(s[v], l), calc(s[u], l));
22         if (b == 1 || (!b && s[v] < s[u])) return v;
23         return u;
24     }
25     upd(u, l, r, s[v]);
26     int mid = (l + r) >> 1;
27     ls[u] = merge(ls[u], ls[v], l, mid);
28     rs[u] = merge(rs[u], rs[v], mid + 1, r);
29     return u;
30 }
```

习题

- [「JSOI2008」Blue Mary 开公司](#)
- [「CodeChef」TSUM2 Sum on Tree](#)
- [「USACO13MAR」Hill Walk G](#)
- [「CF932F」Escape Through Leaf](#)

---

本页面最近更新：2024/10/12 17:40:22, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页贡献者：[StudyingFather](#), [aofall](#), [ChungZH](#), [codewasp942](#), [CoelacanthusHex](#), [do-while-true](#), [Enter-tainer](#), [henrytbtrue](#), [iamtwz](#), [Jelefy](#), [kenlig](#), [leoleoasd](#), [Marcythm](#), [mcendu](#), [Persdre](#), [shuzhouliu](#), [Tiphereth-A](#), [usada-pekora-286](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用